



Programación Concurrente  
Programación Concurrente Multihilos - Java  
Parte V

## Sincronización

Continuamos con los mecanismos de sincronización para la sincronización por cooperación  
Recordemos:

En un escenario concurrente los hilos pueden cooperar o competir por los recursos.  
Cuando compiten por un recurso decimos que es **sincronización por competencia**, y debe trabajarse con mecanismos para lograr la exclusión mutua: semáforos binarios, métodos y bloques sincronizados y locks.  
Cuando cooperan para poder avanzar decimos que es **sincronización por cooperación**, y en esta situación se requiere complementar las facilidades de sincronización para exclusión mutua con mecanismos de espera y notificación.

Una forma de trabajar la sincronización por cooperación es utilizando **semáforos generales**.  
Otra forma es utilizando Monitores, cuya semántica se logra en Java por la utilización de bloqueos intrínsecos, con el patrón **synchronized – wait – notify**. (notifyAll)

Y una forma mas son los locks acompañados de variables de condición ...

## **Locks y variables de condición.**

Ya conocemos los locks explícitos que se trabajan en Java con la interfaz LOCK y una de sus implementaciones *ReentrantLock* (ver recuadro). Los objetos cuyas clases implementan la interfaz Lock funcionan de manera muy similar a los locks implícitos que se utilizan en código sincronizado, de manera que sólo un hilo puede poseer el Lock por vez.

**Cerros-Interfaz LOCK en -java.** Un cerrojo es una llave de acceso a una sección crítica. A diferencia de los locks o llaves “implícitas” disponibles a través del uso del método básico de sincronización (por utilización de *synchronized* en Java), un cerrojo es un lock “explícito”, es decir es una llave de acceso a la sección crítica que debe ser adquirida explícitamente y al finalizar la sección crítica debe ser liberada explícitamente. Para trabajar con cerrojos, Java provee la interfaz LOCK y su implementación REENTRANTLOCK. (ApunteProgConcurrenteParte-III, pag 6)

La clase *ReentrantLock* implementa la interfaz Lock permitiendo que un hilo que requiera un lock que ya tiene pueda volver a entrar a su sección crítica, por ser reentrante.

Los locks permiten mayor flexibilidad y pueden tener asociados varios conjuntos para la espera de los hilos. Entonces para utilizar locks para resolver la sincronización por cooperación se agregan las *variables de condición*, que permiten “**esperar para que ocurra algo**”. Puede ser necesario esperar que una cola no este vacía para remover un elemento, esperar que haya espacio disponible para agregar algo a un buffer, esperar que no haya lectores leyendo, ... Este tipo de situaciones es lo que atienden las *variables de condición*.

- Una variable de condición se asocia a un lock, y un hilo debe mantener el lock para poder esperar sobre la condición. (De forma análoga a lo que sucede con los hilos que necesitan hacer un wait/notify sobre un objeto, que deben hacerlo dentro de un “synchronised”)
- Obtenido el lock se chequea la condición, si es true el hilo continua con lo que debe hacer y cuando corresponde libera el lock, si es false el hilo llama a *await()* sobre la variable de condición, que atómicamente libera el lock y bloquea al hilo sobre esa variable de condición.
- Cuando otro hilo llama a *signal()* o *signalAll()* indicando que hubo cambios sobre la condición correspondiente, el hilo que quedo en espera en un *await()* se desbloquea y vuelve a tomar el lock cuando sea su turno de CPU.
- Las operaciones *await()*, *signal()* y *signalAll()* se aplican sobre una variable de condición particular.
- Un mismo lock puede tener asociadas mas de 1 variable de condición. ¿Sucedo lo mismo cuando se trabaja con locks implícitos y conjuntos de espera?

**IMPORTANTE:** a diferencia del mecanismo de métodos/bloques sincronizados, en el que la adquisición y liberación del lock estan implícitos, en el caso del uso de cerrojos, la adquisición y liberación del lock es explicito, y puede ocurrir en alcances diferentes.

Es decir un lock *cerrojo* puede ser adquirido en un método *empezar()* y ser liberado en otro método *terminar()*

```
|  
    ...  
    Reentrant Lock cerrojo= new ReentrantLock();  
    ...  
  
    ...empezar(){  
        cerrojo.lock();  
        ...  
    }  
  
    ...terminar(){  
        ...  
        cerrojo.unlock();  
        ...  
    }  
}
```

### Problema de los Hamsters: solución con locks y variables de condición

Se muestran las clases *RuedaLock* y *PlatoLock* que son las que sufren mayores cambios respecto



a la solución con monitores.

La clase `RuedaLock` utiliza un `reentrantLock` *llave* para lograr la exclusión mutua a la rueda. No es necesario en este caso trabajar con variables de condición, ya que la única condición es la exclusión mutua.

```
public class RuedaLock {
    private Lock llave = new ReentrantLock(true);
    public void rodar(String nombre) {
        try {
            llave.lock();
            System.out.println (nombre + " empieza a rodar" );
            Thread.sleep((long) Math.random()*2500);
        } catch (InterruptedException ex){ ...}
        finally {
            llave.unlock();
        }
    }
}
```

La clase `PlatoLock` utiliza un `reentrant lock` *accesoComida*, junto con 1 variable de condición *hayLugar*, que en conjunto controlarán el acceso al plato de comida por no mas de un maximo de hilos. Cuando un hamster quiere ingresar a comer, y obtiene el lock, es decir obtiene exclusion mutua sobre la sección crítica guardada por el lock, debe además verificar que se dan las condiciones para ingresar a comer, es decir que hayLugar en el plato. Si las condiciones no se dan, entonces el hilo debe esperar, y liberar el lock para permitir que otros hilos puedan ejecutar trozos de codigo cuidados por el mismo lock. El hilo que espera lo hace en la cola de espera de la variable de condición asociada con el lock: *hayLugar*.

Cuando un hamster termina de comer libera a uno o todos los hilos que estan esperando. Los hilos liberados deben volver a adquirir el lock y verificar la condición.

```
public class PlatoLock {
    private Lock accesoComida;
    private Condition hayLugar;
    private int cantidad;
    private int comiendo;

    public PlatoLock(int maximo){
        accesoComida = new ReentrantLock(true);
    }
}
```



```
// se crea la variable de condición asociada al lock accesoComida
    hayLugar = accesoComida.newCondition();
    comiendo = 0;
    cantidad = maximo;
}

public void empezarAComer(String nombre){
    try {
        accesoComida.lock();
        while (comiendo >= cantidad){
            System.out.println( nombre + " debe esperar
                                para comer");
            hayLugar.await();
        }

        System.out.println( nombre + " empieza a comer");
        comiendo ++;
    } catch (InterruptedException ex){}
    finally {
        accesoComida.unlock();
    }
}

public void terminarDeComer(String nombre){
    accesoComida.lock();
    System.out.println( nombre + " ----- termino de
                        comer");

    comiendo --;
    hayLugar.signalAll();
    accesoComida.unlock();
}
}
```

Un reentrant lock, en conjunto con una variable de condición funciona de la misma forma que el synchronized con wait y notify/notifyAll. La diferencia entre los 2 mecanismos de sincronización es que en el caso del reentrant lock es un lock explícito, que debe pedirse y devolverse, y además que un lock puede tener asociadas varias variables de condición.

### Problema del Productor-Consumidor

Podemos utilizar locks con variables de condición para resolver el problema del *Productor-Consumidor*



Recordemos que al trabajar con monitores se tiene un único conjunto de espera por objeto, entonces en el problema del P-C, al trabajar con un objeto compartido buffer, se tendrá sólo el conjunto de espera del objeto buffer para la espera de los hilos por una condición.

En el problema se dan 2 situaciones de espera:

- cuando un productor quiere poner elementos en un buffer lleno
- cuando un consumidor quiere consumir elementos de un buffer vacío.

Con un único conjunto de espera se da la situación de que todos los hilos que deben esperar lo hacen en el mismo conjunto. Luego cuando se produce una notificación, esta debe hacerse con la orden *notify-all()* para asegurar que se desbloquee al menos 1 hilo del tipo apropiado.

Si en el CE hubiera hilos productores y consumidores bloqueados (que podría ocurrir dependiendo del tamaño del buffer), cuando un hilo productor agrega un elemento debe notificar a un consumidor ya que hay un elemento (al menos) en el buffer para consumir. Si lo hiciera con un *notify* se corre el riesgo de que no sea un consumidor el que se desbloquee y pueda producirse deadlock.

Al utilizar locks con variables de condición, se pueden tener 2 variables de condición, una para los productores y otra para los consumidores, y así notificar solo a un hilo del tipo apropiado. (Sería como tener 2 conjuntos de espera)

```
public class Buffer {
    private int cantidad; // mantiene la cantidad de elementos
                        en el buffer
    private final int tamaño; // tamaño del buffer

    //para exclusión mutua y tener las condiciones asociadas
    private final Lock mutex = new ReentrantLock();

    //para sincronizar los productores sobre mutex
    private final Condition productores;

    //para sincronizar los consumidores sobre mutex
    private final Condition consumidores;

    public Buffer(int tam){
        this.cantidad = 0;
        this.tamaño = tam;
        this.productores = mutex.newCondition();
        this.consumidores = mutex.newCondition();
    }

    public void producir(int id) throws InterruptedException {
        mutex.lock();
        while(cantidad == tamaño){
```

```
        System.out.println("PRODUCTOR A ESPERAR!!!" + id);
        productores.await(); //espera bloqueado
    }

    cantidad ++;
    consumidores.signal(); //notifica a un consumidor
    mutex.unlock();
}

public void consumir(int id) throws InterruptedException {
    mutex.lock();
    while(cantidad ==0){
        System.out.println("CONSUMIDOR A ESPERAR!!" + id);
        consumidores.await(); //espera bloqueado
    }

    cantidad --;
    productores.signal(); //notifica a un productor
    mutex.unlock();
}
}
```

En el ejemplo no se captura la excepción “InterruptedException” requerida por el uso de la operación `await()`, la misma si se produce es propagada hacia afuera, o sea hacia el método llamador. En general cuando se utilizan locks, es una buena práctica contemplar la excepción “InterruptedException” y realizar el “unlock” dentro de un bloque “finally”, para asegurar que el lock sera liberado (ver método *empezarAComer(...)* en el ejemplo de los hamster)

Si bien los cerrojos proveen mayor flexibilidad también agregan mayores posibilidades de error.

Al igual que el mecanismo de semáforos proveen la operación de adquisición NO-BLOQUEANTE `tryLock()`, es decir que el hilo que la ejecuta y no logra adquirir el lock no se bloquea.

## Bibliografia

- **Concurrent Programming in JAVA: design principles and patterns** - Doug Lea
- **Thinking in JAVA** - Bruce Eckel - President, MindView Inc. - 2008
- **Seven Concurrency Models in Seven Weeks**. When Threads Unravel - Paul Butcher - The Pragmatic Programmers Inc. - 2014
- **Java Concurrency in Practice** - Brian Goetz - et all - Addison Wesley 2006.
- **Documentacion Oracle**: <http://docs.oracle.com/javase/tutorial/essential/concurrency/> y el paquete `java.util.concurrent`.
- **Orientación a Objetos con Java y UML** - Carlos Fontella - nueva libreria nl 2004